

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-209БВ-26

Студент: Кравцов М.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.09.26

Москва, 2026

Постановка задачи

Вариант 17.

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процесс child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод.

Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы удаляют все гласные из строк.

Общий метод и алгоритм решения

Вспомогательные функции, написанные для выполнения задачи:

- `int read_filename(char *out_buf, int max_len)` — читает данные из stdin до перехода на новую строку (или до достижения размера строки параметра `max_len`) записывает в `out_buf` прочитанную строку добавляя в её конец терминальный оператор „\0“. Функция возвращает длину прочитанной строки
- `pid_t born_child(int pipe_read_fd, const char *filename)` — создаёт дочерний процесс, для дочернего файла закрывает дескриптор `STDIN_FILENO` и делает его копией `pipe_read_fd`, аналогично с `STDOUT_FILENO` и `file_fd`. Дочерний процесс
- `bool is_vorb(char c)` — проверяет является ли буква гласной
- `int transform(char *str, int len)` — удаляет гласные из строки

Алгоритм:

- считать первые 2 строки из `STDIN_FILENO` с помощью `read_filename`
- создать файлы с таким названием, если они есть очистить их содержимое
- инициализировать дочерние процессы с помощью `born_child`
- в цикле считывать строки из `STDIN_FILENO` и по правилу фильтрации (длина строки меньше 10 в первый файл, иначе во второй файл) отправлять в `pipe1` и `pipe2`
- Дочерние процессы после получения строки удаляют из неё гласные с помощью `transform`
- Дочерние процессы записывают обработанные строки в созданные ранее файлы

Использованные системные вызовы:

- `pid_t fork(void);` — создает дочерний процесс.
- `int pipe(int *fd);` — создает однонаправленный буфер данных

- `ssize_t write(int fd, const void *buf, size_t n)` — запись в файловый дескриптор буфера размером `n`
- `ssize_t read(int fd, void *buf, size_t nbytes)` — читает `nbytes` байтов из дескриптора `fd` в буфер `buf`
- `int open(const char *path, int oflag, ...)` — создаёт дескриптор файла, лежащего по пути `path`
- `int dup2(int fd, int fd2)` — дублирует `fd` в `fd2`, закрывает `fd2` и открывает его в том же файле
- `int execve(const char *path, char *const argv[], char *const envp[])` — заменяет текущий процесс, выполняет `path` с аргументами `argv` и
- `int close(int fd)` — закрывает файловый дескриптор
- `pid_t waitpid(pid_t pid, int stat_loc, int options)` — ждёт завершения дочернего процесса `pid`

Код программы

main.c

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#define BUFFER_SIZE 256

int read_filename(char *out_buf, int max_len) {
    int bytes_read = 0;
    char ch;

    while (bytes_read < max_len - 1) {
        int res = read(STDIN_FILENO, &ch, 1);

        if (res <= 0) {
            break;
        }

        if (ch == '\n') {
            break;
        }
        out_buf[bytes_read] = ch;
        bytes_read++;
    }

    out_buf[bytes_read] = '\0';
    return bytes_read;
}
```

```

pid_t born_child(int pipe_read_fd, const char *filename) {
    pid_t pid = fork();

    if (pid < 0) {
        write(STDERR_FILENO, "Failed to fork\n", 15);
        return -1;
    }

    if (pid == 0) {
        int file_fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (file_fd < 0) {
            write(STDERR_FILENO, "Failed to open file\n", 20);
            _exit(1);
        }

        dup2(pipe_read_fd, STDIN_FILENO);
        dup2(file_fd, STDOUT_FILENO);

        close(pipe_read_fd);
        close(file_fd);

        char *const args[] = {"/child", NULL};
        char *const envp[] = {NULL};
        execve("/child", args, envp);

        write(STDERR_FILENO, "Failed to execve\n", 17);
        _exit(1);
    }
    return pid;
}

int main() {
    char file1[BUFFER_SIZE];
    char file2[BUFFER_SIZE];

    int len1 = read_filename(file1, BUFFER_SIZE);
    int len2 = read_filename(file2, BUFFER_SIZE);

    int pipe1[2], pipe2[2];
    pipe(pipe1);
    pipe(pipe2);
    pid_t child1 = born_child(pipe1[0], file1);
    pid_t child2 = born_child(pipe2[0], file2);
    close(pipe1[0]);
    close(pipe2[0]);
}

```

```

char buffer[BUFFER_SIZE];
int bytes_read;

while ((bytes_read = read_filename(buffer, BUFFER_SIZE)) > 0) {
    buffer[bytes_read] = '\n';
    int total_bytes = bytes_read + 1;

    if (bytes_read > 10) {
        write(pipe2[1], buffer, total_bytes);
    } else {
        write(pipe1[1], buffer, total_bytes);
    }
}

close(pipe1[1]);
close(pipe2[1]);

waitpid(child1, NULL, 0);
waitpid(child2, NULL, 0);

return 0;
}

```

child.c

```

#include <stdbool.h>
#include <unistd.h>

#define BUFFER_SIZE 256

static const char VERBS[] = {'a', 'e', 'i', 'o', 'u', 'y',
                              'A', 'E', 'I', 'O', 'U', 'Y'};
static const int VERBS_COUNT = sizeof(VERBS) / sizeof(VERBS[0]);

bool is_verb(char c) {
    for (int i = 0; i < VERBS_COUNT; i++) {
        if (c == VERBS[i]) {
            return true;
        }
    }
    return false;
}

int transform(char *str, int len) {
    int write_index = 0;

    for (int read_index = 0; read_index < len; read_index++) {
        if (!is_verb(str[read_index])) {

```

```

        str[write_index] = str[read_index];
        write_index++;
    }
}

return write_index;
}

int main() {
    char buffer[BUFFER_SIZE];
    int bytes_read;

    while ((bytes_read = read(STDIN_FILENO, buffer, BUFFER_SIZE)) > 0) {
        int new_length = transform(buffer, bytes_read);
        write(STDOUT_FILENO, buffer, new_length);
    }
    return 0;
}

```

Протокол работы программы

ТЕСТ 1

```

$ ./main
file1
file2
Aboba
RustamKrutoi228
ILoveLinuxLMAO
TimurKachok777
LATEX
LOL
$ cat < file1
bb
LTX
LL
$ cat < file2
RstmKrt228
LvLnXLM
TmrKchk777

```

ТЕСТ 2

```

$ ./main
LessThan10
Other
RandomLettersInDifferentOrder
HggDSDADFDBBFSKJDBJXc

```

oeiwiosdisdiadsd
Okiada
NotRandomLetters
JoJoIsMyFavoriteAnime
You Also Can Write Strings With Space
rm -A home
But you can't write in different languages
PludFace :(
\$cat < LessThan10
kd
rm - hm
\$cat < Other
RndmLttrsDffrntrdr
HgqDSDDFDBBFSKJDBJXc
wsdsddsd
NtRndmLttrs
JJsMFvrtnm
Is Cn Wrt Strngs Wth Spc
Bt cn't wrt n dffrnt lnggs
PldFc :(

Вывод

Навыки приобретенные в процессе выполнения данной работы: управление процессами (fork, execve), организация IPC через pipe и перенаправление потоков (dup2). Одна из сложностей: невозможность использовать scanf и printf по условиям лабораторной, чтобы заменить эти функции в процессе выполнения лабораторной работы были использованы write и read.